

Introduction to the GNU GMP Library

Diyora Salimova

ETH Zurich

Abstract

The aim of the report is to discuss one of the most popular libraries of the GNU, namely, the GMP. Firstly, we give an introduction to the library. Then we explain the basic rules on how to use it and give common properties of its functions. Finally we consider some points on its memory handling and efficiency.

1. Introduction

GMP stands for GNU Multi Precision library and it is a part of the GNU project. Let's start with some introduction to the GNU project.

The GNU project is a free software and mass collaboration project, publicized on the 27th of September 1983, by popular and talented programmer, freedom activist Richard Stallman at MIT. The main goal of this project was to give the opportunity to computer users to develop their software collaboratively, helping each other. Users of the GNU have following freedom rights:

- run software of the project,
- copy and distribute it,
- study it,
- modify it.

GNU guarantees these freedom rights legally via its licenses. Since its launching numerous useful libraries were developed under it, one of them being GMP.

GMP is a popular library that gives users capability to operate with arbitrary precision integers, rational numbers and floating point numbers.

It was written in 1991 by Torbjorn Granunland who is still considered to be the main developer. However several other individuals and organizations have significantly contributed GMP. Originally the library was written in C language, though extensions to other common languages were made. It is free meaning that everyone is free to use and to redistribute it on a free basis. However there are some restrictions indicated in its license GNU LGPL. For example, users of the library should give all the rights that they have while redistributing copies of the GNU GMP, including all the source codes. This restriction is made to protect the users on the first case. The other one is that recipients should know about all of the modifications that were made, so that they don't get impression what they got it originally from the developers of the library. Otherwise that could hurt reputation of the developers.

2. Why to use the GMP?

The main reason is that computers usually process numbers of a maximum 32 or 64 bits at a time. That means if you for example write a program in C and declare `int x`, then `x` is a 32 bit number or 64 bits. Any attempt to add one to the maximum `x` wraps around to the minimum integer.

Let's consider simple example where we want to compute factorial of a natural number in C program:

```
int fact(int n){
    int i;
    int p = 1;
    for (i=1; i <= n ; ++i){
        p = p * i;
    }
    return p;
}

int main(int argc, char * argv[]){
    int n;
    n = atoi(argv[1]);
    assert( n >= 0);
    /* Print the factorial */
    printf ("%d ! = %d \n", n, fact(n));
    return 1;
}
```

If we try to compute factorial of very small numbers the program works, but if we for example enter 20 as an argument it gives us -2102132736, a negative number!

If that would be a case, it would emerge problems in many industries which have to work with very large numbers and high precisions, like banks, insurance companies, and so on.

GMP is basically a solution for this kind of problems. It allows us to use numbers whose sizes can grow dynamically to the required precision. Some important features of GMP are:

- It uses full words as the basic arithmetic type.
- It uses different algorithms for different operand sizes, since algorithms that are faster for very big numbers are usually slower for small numbers.
- It uses highly optimized assembly language code for the most important inner loops and specialized for different processors.

By putting many words together, GMP can support 128, 256 or 1024 bits. The library can dynamically allocate memory for accommodating extra bits of precision as and when needed.

3. Installing and using the GMP

The library can be downloaded from its official web-page <https://gmplib.org/>. Download file includes include GMP compiled static library files gmp.h, gmp.lib as well as GMP library user manual. There also different examples to help users to get basic ideas on how to use it. After download you have to compile library. Usually it takes a big amount of time, but the functionality of the library is really worth it. Once you are done you can use it adding to your code

```
#include <gmp.h>
```

To compile your code you must link it against 'libgmp' library. On a Unix-like system it is done with

```
gcc myprogram.c -lgmp
```

When you just start to use the GMP library, it might be easy to get scared from the variety of functions. The latest help documentation in English that can be found in the official website would be the best help in that case. In fact, many functions' names are very intuitive and represent their assignments. GMP's function name usually looks like:

`mpz_funxx_dataxx`

Here **funxx** represents function's assignment and **dataxx** indicates the data type that is used within this function.

For convenience the GMP functions are divided in several categories:

- **Integer Functions:** Functions for signed integer arithmetic, begin with `mpz_`(about 150 functions). The associated type is `mpz_t`.
- **Rational Functions:** Functions for rational number arithmetic, begin with `mpq_`(about 40 functions). The associated type is `mpq_t`.
- **Floating-point Functions:** Functions for floating-point arithmetic, begin with `mpf_`(about 60 functions). The associated type is `mpf_t`.
- **BSD Compatible Functions:** Functions compatible with Berkeley MP. The associated type is `MINT`.
- **Low-level Functions:** Fast low-level functions for natural numbers arithmetic, begin with `mpn_`(about 30 functions). The associated type is `mp_limb_t`.
- **Miscellaneous Functions.**

Some common functions from these categories are:

```
mpz_t t; // this is the type of statement is needed to declare a variable
mpz_init (t); // initialization of a variable
mpz_set_ui (t, 2); // assignment of 2 to already initialized variable t
mpz_set_str (t, "1234"); // string assignment
mpz_add (t, a , b); // a + b is assigned to t, that is, t = a + b
mpz_sub (t, a, b); // subtraction
mpz_mul (t, a, b); // multiplication
gmp_printf ("% Zd ", t); // print output t
```

The user also has to remember to initialize the GMP variable by calling one of the special initialization functions before assigning it. Once the variable is not needed it should be cleared out, using one of the functions for that purpose. Therefore the structure of the GMP code would look like

```
#include <gmp.h>
/* ... */
mpz_t n;
mpz_init(n);
/* ... */
mpz_set_ui(n,0);
/* ... */
mpz_clear(n);
```

Here

- **mpz_init(n)** calls the **init** function and tells it to properly initialize **n**.
- **mpz_set_ui(n,0)** asks GMP to set the number **n** to **0**.
- **mpz_clear(n)** asks GMP to deallocate the memory that it has stored for **n**.

4. Memory handling and remarks on efficiency

GMP uses different memory management for different types of variables. Variables of type **mpz_t** and **mpq_t** use very small amount of space. They contain only couple of sizes and pointers allocated to data. After a variable is initialized additional space is allocated whenever it doesn't have enough. However, the reallocation procedure might be costly. Therefore if for example one uses a variable to hold successively increasing values the best way is to allocate the necessary space from the beginning using the functions like **mpz_init2** or **mpz_realloc**. Though, in that case it is not always easy to estimate final size of the variable. In contrast, variables of type **mpf_t** use fixed amount of space which is allocated at the initialization, and the size doesn't change during the algorithm.

Also note that even though GMP tries to be as efficient as possible there is always a trade-off between the efficiency and the used space. Therefore the time used for small operands might be significant compared to the actual

calculation. Last but not least, one has to be careful with the initializing and clearing the variables. Although these procedures are important and sometimes unavoidable they are usually very time consuming. Hence try to avoid unnecessary initializing and clearing. As showed in the code structure in the previous section initialize near the start of a function and clear near the end.

5. Conclusion

GMP is a very powerful library to work with arbitrary precision numbers. It has variety of functions and is also widely used in development of the other libraries of the GNU project. Anyone would be fascinated from the efficiency and the usefulness of the library just from the first try. Yet for the users who want to apply it for more complicated problems it is essential to understand the algorithms behind. They could improve the efficiency and the space allocation using specific functions of the library depending on the given problem.

6. References

- [1] Torbjorn Granlund and the GMP development team (2010),
The GNU Multiple Precision Arithmetic Library, *Manual*, Edition 5.0.1.
- [2] Wikipedia: The Free Encyclopedia.
Wikimedia Foundation, Inc. October 10, 2014
http://en.wikipedia.org/wiki/GNU_Multiple_Precision_Arithmetic_Library.
- [3] GMP official website. October 10, 2014, <https://gmplib.org/>.
- [4] Courses Taught By Sriram Sankaranarayanan.
Tutorial on GMP. October 10, 2014,
http://www.cs.colorado.edu/~srirams/classes/doku.php/gmp_usage_tutorial.